NCollection ERP — Sprint Schedule & Parallelization Plan

> **Version**: 1.0 **Date**: July 16, 2026 **Purpose**: The concrete resource-constrained schedule for Phases 1–3 (through first production deployment). It answers three questions the task tables in DELIVERABLE_1_SYSTEM_DESIGN.md leave implicit: (1) which tasks run **in parallel** vs **in series**, (2) **when** each task starts, and (3) **how idle gaps are filled** by pulling later-phase work forward. Sprints are 2 weeks / 10 working days.
>
> **Companion**: DELIVERABLE_2_TIMELINE_AND_TOOLING.md (sprint ceremonies, velocity), DELIVERABLE_1_SYSTEM_DESIGN.md (task detail + dependencies).

---

1. Load Philosophy — Deliberately Uneven

This is a **backend- and database-heavy platform**: the product is the SaaS layer (provisioning, isolation, pooling, PITR, licensing, APIs) far more than UI. The workload is therefore **intentionally weighted toward DEV-1**, and the developers are **not** loaded equally. This is a design choice, not an imbalance to correct.

Developer	Whole-project dev-days	Share	Concentrated in
DEV-1 (Backend & Infra)	140	**38%**	P2 SaaS automation (31d), P10 HA/scaling/security (23d), P8 REST API (15d)
DEV-2 (Odoo & Business Logic)	117	31%	P1 licensing (21d), P2 billing/lifecycle (22d), P3 UAE localization (19d)
DEV-3 (Frontend & Integration)	111	30%	P1 branding/dashboard/E2E (22d), P7 mobile (14d), P4 dashboards (10d)

Consequence: DEV-3 has lighter periods during the backend-dominated phases (notably Phase 2). This is expected and healthy — those gaps are **filled by pulling frontend-heavy Phase 3/4 work forward** (Arabic translations, PDF invoice templates, dashboards), which the schedule below does explicitly. DEV-3 is never artificially padded with backend work they aren't suited for.

---

2. Parallel vs Series — The Structure

2.1 What runs in parallel (independent chains, different devs)

Three chains run **concurrently from Day 1** with zero cross-blocking:

```
DEV-1 (infra chain, series):  P1-T02 → P1-T03 → P1-T06 → P1-T19 → P1-T21
DEV-2 (licensing chain, series): P1-T01 → P1-T07 → P1-T09 → P1-T10 ┐
DEV-3 (branding chain, series):  P1-T13 → P1-T14 → P1-T16          │
                                                                   ▼
                                          all converge → P1-T21 (gate)
```

- **DEV-1's infra tasks are strictly serial** — each configures the layer the next needs (config → proxy → routing → auth → gate). No way to parallelize within one dev.
- **The three chains are mutually parallel** — DEV-1's Nginx work and DEV-2's role definitions share no dependency, so they proceed simultaneously.
- **P1-T04 (OCA deps) and P1-T05 (CI) are DEV-1 "filler"** — independent of the infra chain, slotted into DEV-1's early gaps.

2.2 What must run in series (true dependencies)

Series chain	Why it cannot parallelize
P1-T07 → P1-T09 → P1-T10	Licensing enforcement needs the module-list field (T07), then the menu engine (T09) before ORM enforcement (T10) can map allowed models
P1-T02 → P1-T03 → P1-T06	Routing (T06) needs the proxy (T03) which needs the config (T02)
P1-T08 → P1-T11 → P1-T12	Owner settings (T12) needs Apps-stripping (T11) which needs the role groups (T08)
P2-T01 → P2-T02 → P2-T03	Config sync (T03) needs the pipeline (T02) which needs the engine (T01)
P2-T04 → P2-T05	Per-tenant backups build on the PITR/WAL foundation

2.3 The single longest chain (critical path)

```
P1-T01(1) → P1-T07(5) → P1-T09(4) → P1-T10(3) → P1-T21(3) = 16 days
```

No amount of extra developers shortens Phase 1 below ~16 working days — this is the theoretical floor. DEV-2 owns this chain, which is why DEV-2's Phase 1 load (21d) is the binding constraint of the phase, not DEV-3's (22d — but more parallelizable).

---

3. The Merged Schedule (Phases 1–3 → First Production Deploy)

Resource-constrained (each dev serial), dependency-honoring, cross-phase pull-forward enabled. **~71 working days ≈ 7 sprints ≈ 14 weeks** of pure development (add review/QA/buffer → the 8–12 week Phase-1 + go-live-at-~month-4 figures in DELIVERABLE_2).

Sprint	DEV-1 (Backend/Infra)	DEV-2 (Odoo/Logic)	DEV-3 (Frontend)

S1 (wk 1–2)	P1-T04, P1-T02, P1-T05, P1-T03, P1-T06 *(OCA deps, config, CI, Nginx, routing)*	P1-T01, P1-T07, P1-T08, P1-T18 *(skeletons, models, roles, email)*	P1-T13, P1-T17 *(branding, dashboard)*
S2 (wk 3–4)	P1-T15, P1-T19 *(URL rewrite, auth hardening)*	P1-T09, P1-T11, P1-T10 *(menu vis, stripping, ORM enforcement)*	P1-T14, **P2-T16**, P1-T16 *(login, checkout pulled fwd, dyn branding)*
S3 (wk 5–6)	**P1-T21 gate**	P2-T11, P2-T15 *(billing, admin dash)*	P1-T20, P1-T12, **P3-T08** *(E2E, owner settings, Arabic pulled fwd)*
S4 (wk 7–8)	P2-T04, P2-T01, P2-T07 *(PITR, provisioning runner, staging)*	P2-T12, P2-T13, P2-T14 *(lifecycle, Stripe, dunning)*	*(light — pull P4-T03 CEO dashboard prototype fwd, or E2E journeys)*
S5 (wk 9–10)	P2-T06, P2-T05, P2-T02 *(domains, backups, auto-provision)*	P3-T04 *(UAE VAT)*	P2-T17, **P3-T09** *(email automation, PDF invoices)*
S6 (wk 11–12)	P2-T10, P2-T08, P2-T09, **P2-T18 gate** *(monitoring, hardening, PgBouncer, gate)*	P2-T03, P3-T07 *(config sync, approvals)*	*(light — P3/P4 UI polish, translation QA)*
S7 (wk 13–14)	P3-T02, P3-T12, P3-T03, **P3-T13 go-live** *(perf tuning, security assessment, load test, DEPLOY)*	P3-T01, P3-T05, P3-T06, P3-T11 *(OCA verify, CoA, currency, import)*	P3-T10 *(MIS reports)*

Bold = **phase gate** or **task pulled forward from a later phase to fill an idle window**.

3.1 Utilization (this proves the parallelism is real)

Developer	Working days used / 71	Utilization	Reading
DEV-1	60	**84%**	Near-continuous — the sustained critical resource, as intended
DEV-2	62	**87%**	Highest — owns the Phase-1 critical path + Phase-2 business logic
DEV-3	42	**59%**	Lighter by design; gaps filled with pulled-forward frontend work (P2-T16, P3-T08/09)

DEV-3's 59% is the honest number. Rather than invent backend busywork, S4 and S6 are explicitly available for: pulling **Phase 4 dashboards** forward (they only depend on P1-T07, already done), deepening the **E2E suite**, or **Arabic RTL QA** — all frontend work that is genuinely ready and valuable early.

---

4. How Idle Gaps Are Filled (Cross-Phase Pull-Forward)

The key to keeping a backend-weighted team productive is letting the lighter-loaded dev pull *ready* future work forward. A task is "pullable" when **all its dependencies are already complete**, regardless of its phase number.

Idle window	Dev	Pulled-forward work	Why it's ready

S2	DEV-3	**P2-T16** Self-service checkout	Only depends on P1-T13 (branding), done in S1
S3	DEV-3	**P3-T08** Arabic/English translation	Only depends on P1-T13; independent of all Phase 2
S4	DEV-3	**P4-T03** CEO Dashboard (prototype) or extra E2E journeys	P4 depends only on P1-T07/aggregation; frontend can start UI shell early
S5	DEV-3	**P3-T09** UAE PDF invoices	Depends on P3-T04 (VAT), completed by DEV-2 in S5
S6	DEV-3	Phase 3/4 UI polish, translation QA	No hard dependency; buffer for gate stabilization

Rule: before a lighter-loaded dev pulls a task forward, confirm on the GitHub issue that its dependencies are `Done`. Never pull a task whose prerequisites are still open — that recreates the idle-wait the pull was meant to avoid.

---

5. Later Phases (4–10) — Load Rhythm

Beyond go-live the same principle holds: the load oscillates so that when one dev spikes, others have pull-forward room.

Phase	D1	D2	D3	Who leads	Lighter devs pull forward
4 Dashboards	4	3	**10**	DEV-3	DEV-1/2 → start Phase 6 portal backend / Phase 8 API design
6 Portal	5	**8**	9	DEV-2/3	DEV-1 → Phase 8 REST API foundation
5 AI	**12**	9	9	DEV-1	balanced
7 Mobile	8	9	**14**	DEV-3	DEV-1/2 → Phase 8 endpoints/webhooks
8 Platform	**15**	9	8	DEV-1	DEV-3 → API docs portal, integration UI
10 Enterprise	**23**	9	10	DEV-1	DEV-2/3 → Phase 9 marketplace prep (if greenlit)

DEV-1 remains the highest-loaded across the project (38% overall) — correct for a platform whose value is its backend. DEV-3 peaks only in the UI-heavy phases (4, 7). This is the intended shape.

---

6. Scheduling Assumptions & Caveats

- **Dev-days are implementation only.** Real calendar time adds code review, PR turnaround, meetings, and iteration — DELIVERABLE_2 applies the ~1.5× multiplier that turns 71 dev-days into the 8–12 week Phase-1 window.
- **The schedule is a plan, not a contract.** Estimates carry uncertainty; the sprint board (GitHub Projects) is the live source of truth. Re-derive this schedule if estimates change materially.
- **Gate tasks (P1-T21, P2-T18, P3-T13) are hard synchronization points** — no later-phase work merges until the gate's regression + isolation suite is green.

- **This models Phases 1–3 precisely** (the path to revenue); Phases 4–10 are shown as load rhythm only, to be scheduled in detail at each phase kickoff.

> **Document End** · Regenerate the schedule whenever task estimates or assignments change. The numbers here were computed by resource-constrained list scheduling over the DELIVERABLE_1_SYSTEM_DESIGN.md dependency graph.